



WEEK ONE

在第一週，我們會一起完成 **App #1 | 微型 CRM**。它不是因為你要當業務，而是因為**所有**產品，幾乎都從這一套能力開始。

1. 把模糊的想法，寫成清楚的 Spec

不再只是「我想做一個 app」，而是能說清楚它要做什麼、資料長什麼樣、流程怎麼跑。

2. 定義資料結構 (Schema)

學會用產品與資料的角度思考，而不是從畫面或感覺開始。

3. 拆解使用流程 (User Flow)

讓每一個按鈕、每一次互動，都有明確的前後順序。

4. 用對的 Prompt，讓 AI 照規格做事

不是祈禱 AI 猜中你要什麼，而是清楚告訴它「你只能照這樣做」。

5. 完成第一個能跑的互動網站

表單可以新增資料，資料真的會存進雲端，畫面會即時更新。

6. 第一次理解：原來我真的做得到

不是 Demo，不是玩具，而是一個「活著的系統」。

CHAPTER 1 你現在在學的不是寫程式	8
1.1 先講清楚一件事：這堂課不是在教你寫 code	8
1.2 為什麼 Vibe Coding 這麼容易撞牆？	9
1.3 正確的 Vibe Coding 定義	10
1.4 為什麼我們一直在講「文件」？	11
1.5 什麼是 PRD / SDD / DDD ？一張表讓你一次看懂	12
1.6 為什麼 SDD 是 Vibe Coding 的生死線？	13
1.7 那 SDD 到底長什麼樣子？（先給你一個感覺）	14
1.8 這一章學完，你現在站在哪？	15
下一章你會學什麼？	15
CHAPTER 2 為什麼第一週一定要先寫 SPEC？	16
2.1 我們先誠實面對一件事：你腦中現在其實是亂的	16
2.2 什麼是 Spec ？	16
2.3 為什麼不先寫 Spec ， Vibe Coding 一定會歪？	17
2.4 在這門課 Spec 的實際用途是什麼？	18
2.5 這點一定要釐清： Spec ≠ 一大本文件	18
2.6 為什麼 Spec 要在「寫任何程式之前」完成？	19
2.7 本課程使用的 Spec 種類定位（再對照一次）	19
2.8 一個錯誤觀念，現在就要先拆掉	20
2.9 這一章學完，你應該能做到什麼？	20
下一章你會學什麼？	20
CHAPTER 3 像產品經理一樣整理腦中想法	21
3.1 先打破一個迷思：你不是要「設計一個完整產品」	21

3.2 產品思維的第一個關鍵能力：降維思考	22
3.3 所有產品的第一個功能，長得其實都一樣	22
3.4 從「產品想法」到「第一個功能」的拆解步驟	23
Step 1 用一句話描述你的產品 (PRD 極簡版)	23
Step 2 問自己一個殘酷但有效的問題	23
Step 3 把「功能」翻譯成「行為描述」	24
3.5 現在，你的第一個功能應該長這樣	24
3.6 為什麼我們不急著想第二個、第三個功能？	25
3.7 把第一個功能，轉成「Spec 草稿」	26
3.8 這一章學完，你現在站在哪？	26
下一章你會學會什麼？	26
CHAPTER 4 你的產品資料骨架：SCHEMA	27
4.1 為什麼我們現在才開始用 AI？	27
4.2 什麼是 Schema	27
4.3 為什麼 Schema 沒定義，AI 一定會亂寫？	27
4.4 Week 1 的 Schema 設計原則 (請照做)	28
4.5 範例：第一個功能的 Schema (示意)	28
學員專用指令區塊 Schema 生成	29
4.6 你要學會檢查 AI 的 Schema (很重要)	30
學員專用指令區塊 Schema 檢查 (必用)	30
4.7 本章你應該完成的成果	31
下一章你會學什麼？	31
CHAPTER 5 讓你的系統「動起來」：USER FLOW	32
5.1 為什麼資料定義完，還不能直接寫程式？	32
5.2 什麼是 User Flow？	32

5.3 為什麼沒有 User Flow · Vibe Coding 一定失控？	33
5.4 Week 1 的 User Flow 撰寫原則（請照做）	34
5.5 範例：微型 CRM 的 User Flow（Create + Read）	34
5.6 把你的 Flow 寫出來	35
學員專用指令區塊 User Flow 拆解	37
5.7 Flow 寫完後，你一定要自己檢查的三件事	38
學員專用指令區塊 Flow 檢查（防止 AI 亂補）	38
5.8 本章完成後，你現在擁有什麼？	39
下一章你會學什麼？	39
CHAPTER 6 不是聊天，是規格：PROMPT ENGINEERING	40
6.1 為什麼你以前下的 Prompt · AI 都「做得很怪」？	40
6.2 在這門課裡，Prompt 的正確定位是什麼？	40
6.3 為什麼要先學「結構化 Prompt」？	41
6.4 Week 1 使用的 Prompt 結構（固定版）	41
6.5 AWS Q Developer × Copilot 的角色分工	42
學員專用指令區塊 建立工程協作角色（必做）	43
6.6 把 Schema + Flow 餵給 AI（但不叫它寫 code）	44
學員專用指令區塊 Spec 對齊確認	44
6.7 為什麼我們要「一個任務一個 Prompt」？	45
學員專用指令區塊 單一任務實作 Prompt（核心）	45
6.8 Go Live Server：你一定要「邊做邊看」	46
6.9 本章你現在應該具備的能力	46
下一章你會學什麼？	46
CHAPTER 7 實作第一個可跑的功能（CREATE）	47
今天的「極簡 CRM」到底包含什麼？	47

7.1 現在我們終於要開始「真的做東西」了	48
7.2 Week 1 的實作目標 (請牢記)	48
7.3 專案初始化 (VS Code)	49
學員專用指令區塊 專案初始化協作指令	49
7.4 建立輸入介面 (表單)	50
學員專用指令區塊 建立輸入表單 (Create UI)	50
7.5 用 Go Live Server 驗證畫面	51
7.6 寫入資料的邏輯 (Create Logic)	52
學員專用指令區塊 新增資料邏輯 (Create)	52
7.7 再次 Go Live：確認資料真的有被新增	53
7.8 本章完成後，你現在擁有什麼？	53
下一章你會學什麼？	53
附錄：DEBUG	54
D-1 先說一句很重要的話	54
D-2 新手最常見的 5 種壞法	55
D-3 Week 1 Debug 的唯一正確心法	57
學員專用指令區塊 定位問題	57
學員專用指令區塊 局部修正 (救命指令)	58
學員專用指令區塊 Flow 回溯檢查 (防止亂修)	59
D-4 什麼情況下，你可以說「我這一關過了」？	59
CHAPTER 8 把資料顯示成列表：READ	60
8.1 為什麼 Read 這一步這麼重要？	60
8.2 Week 1 的 Read 在做什麼 (請不要擅自加戲)	60
8.3 什麼是 Read 的本質：不是畫面，是「資料流動」	61
8.4 Read 的基本流程 (人話版)	61
學員專用指令區塊 建立資料列表容器	61

8.5 讀取資料的邏輯 (Read Logic)	62
學員專用指令區塊 讀取資料邏輯 (Read)	62
8.6 當新增資料後，列表要怎麼更新？	63
學員專用指令區塊 新增後刷新列表 (穩定版)	63
8.7 Go Live Server：確認「完整流程」跑通	64
8.8 本章完成後，你現在站在哪？	64
在進入下一章之前，你需要先知道一件事	65
下一章你會學什麼？	65
CHAPTER 9 你現在擁有的是「可擴充的雛形」	66
9.1 先幫你正名一件事	66
9.2 回顧你在 Week 1 真正完成的事情	66
9.3 你現在「刻意沒有做」的事情 (這是好事)	67
9.4 今日最常見的技術債來源 (請避開)	67
9.5 Week 1 正確的進階姿勢 (先知道就好)	68
9.6 如果你現在想繼續改，請先問自己三個問題	69
學員專用指令區塊 技術債風險檢查	69
9.7 這一章你現在應該帶走的觀念	70
下一章你會學什麼？	70
CHAPTER 10 課後作業與個人產品延伸	71
10.1 先說清楚：這不是交作業，是產品啟動	71
10.2 課後作業的真正目的	71
10.3 課後作業內容說明 (請一步一步來)	72
作業任務一 定義你的「第一個功能」	72
作業任務二 完成你的 Spec (文字版)	72
作業任務三 實作 Create + Read	73
學員專用指令區塊 個人產品 Spec 啟動指令	74

10.4 課後六天，你該怎麼安排才不會爆掉？	75
學員專用指令區塊 卡關救援指令	76
10.5 什麼狀況下，你可以安心進入 Week 2？	77
10.6 Week 1 的真正收穫（請帶走這句話）	77
下一步，你會在 Week 2 做什麼？	77

Chapter 1 你現在在學的不是寫程式

是「怎麼把想法變成 AI 能執行的規格」

1.1 先講清楚一件事：這堂課不是在教你寫 code

在開始之前，我想先幫你校正一個很重要的認知。如果你是抱著下面任何一個期待來的，請放心，你沒有來錯，但我們要稍微調整一下期待的方向：

- 我想學怎麼用 AI 寫網站
- 我想靠 Vibe Coding 快速做出一個 App
- 我不會寫程式，但想做自己的產品

這些都**沒有問題**，但這堂課真正要教你的，不是怎麼打程式碼，而是這件事：

當你腦中有一個想法時

你有沒有能力把它整理成

讓 AI 可以『照著執行』、而不是『自由發揮』的內容

如果你沒有這個能力，AI 再強，你最後都會遇到同一種結局：

- 看起來很厲害
- 但你不敢改
- 一改就壞
- 壞了不知道為什麼
- 最後放棄

這不是技術問題，是**思考結構問題**。

1.2 為什麼 Vibe Coding 這麼容易撞牆？

我們先誠實面對現況。

多數人第一次使用 Vibe Coding，流程大概是這樣：

1. 打開 ChatGPT / Copilot / Q Developer
2. 說一句話：「幫我做一個 XXX 系統」
3. AI 生出一大包程式碼
4. 看起來很厲害，但不知道從哪裡開始看
5. 想改一點東西 → 整包壞掉
6. 開始懷疑人生

這個過程中，**AI 沒有做錯事**。

真正發生的問題是：你把「工程決策權」全部丟給了 AI，但 AI 並不知道你真正想要的是什麼，所以它只能「猜」。

1.3 正確的 Vibe Coding 定義

在這門課裡，我們對 Vibe Coding 有一個非常明確的定義：

Vibe Coding = 人負責規格，AI 負責實作

也就是說，角色分工是這樣的：

角色	負責的事情
你（人）	想清楚要做什麼、怎麼做、做到哪
AI	把你說清楚的東西，轉成程式碼

如果你沒有先「想清楚」，AI 就一定會亂做，不是因為它不聰明，而是因為你沒有給它可以依循的結構。

1.4 為什麼我們一直在講「文件」？

你可能會在這堂課一直聽到這些名詞：

- PRD
- SDD
- DDD
- Spec
- Schema
- Flow

如果你心裡出現一句話：「我不是工程師，為什麼要看這些？」，這是非常正常的反應。

所以我們現在就來一次把這些名詞「**定位清楚**」，你就會知道：

- 哪些是你現在一定要會的
- 哪些是你現在只要「知道存在就好」

1.5 什麼是 PRD / SDD / DDD ? 一張表讓你一次看懂

下面這張表，請你先慢慢看，不用急著全部記住。

文件角色對照表

文件	全名	解決的問題	給誰看的	在本課程的定位
PRD	Product Requirement Document	這是什麼產品？為誰做？解決什麼問題？	人，PM，老闆，團隊	要會「講得出來」即可
SDD	Specification-Driven Development	功能怎麼做？資料怎麼存？流程怎麼跑？	AI，工程師	Week 1 核心能力
DDD	Domain-Driven Design	資料與概念要怎麼分區、不混亂	工程師	先理解「精神」，不用精通

你現在只需要記住一句話：

PRD 是「對人說的產品說明」

SDD 是「對 AI 說的操作說明書」

而這門課，會花最多力氣在 SDD 上。

1.6 為什麼 SDD 是 Vibe Coding 的生死線？

因為 AI 寫程式的方式，跟人完全不一樣，人可以「腦補」，AI 不行。

因此如果你只跟 AI 說：「我要一個客戶管理系統」，它一定會幫你補一堆你沒想過或還沒想要做的東西：

- 欄位自己加
- 流程自己排
- UI 自己設計
- 邏輯自己猜

結果就是：它做的不是你要的那個產品

SDD 的存在，就是為了避免這件事發生。

1.7 那 SDD 到底長什麼樣子？（先給你一個感覺）

不用急，後面幾章會拆得非常細。你現在只要知道：一份最基本的 SDD，一定回答三件事

1. 資料是什麼樣子（Schema）
2. 使用流程怎麼跑（User Flow）
3. 這個功能做到哪裡就好（Scope）

舉例（先感覺就好）：

Schema：

- name
- email
- phone

Flow：

1. 使用者填寫資料
2. 點擊新增
3. 系統儲存
4. 顯示在列表

這種「看起來很樸素」的東西，才是 AI 最需要的內容。

1.8 這一章學完，你現在站在哪？

如果你現在能回答下面三個問題，代表你這一章已經學會了：

1. 我現在不是在學「寫程式」，而是在學「寫規格」
2. PRD / SDD / DDD 的用途我分得出來
3. 我知道為什麼「直接叫 AI 做產品」一定會出事

這樣就夠了。

下一章你會學什麼？

下一章 (Chapter 2)，我們會開始做一件非常重要的事：

教你像產品經理一樣，把腦中的混亂想法，整理成「第一份 Spec」

你會開始真的動手寫，而不是只是理解概念。

Chapter 2 為什麼第一週一定要先寫 Spec ?

不先寫，後面一定還債

2.1 我們先誠實面對一件事：你腦中現在其實是亂的

如果我現在問你：「你想做的產品是什麼？」

你很可能會回答其中一種：

- 一個可以幫我管理 XXX 的系統
- 一個有點像 Notion / Trello / 某某 App 的東西
- 反正就是一個可以用的工具

這些回答都沒有錯，但它們有一個共同的問題：它們都還不能拿來開發

不是因為你想得不夠好，而是「產品想法」跟「可開發規格」中間，本來就隔著一道牆。

Spec，就是用來跨過這道牆的工具。

2.2 什麼是 Spec ?

在這門課裡，我們對 Spec 的定義非常簡單：

Spec = 把模糊的想法，轉成「可以一步一步照做」的說明

請注意幾個關鍵字：

- 不是寫漂亮文件
- 不是寫給工程師炫技
- 不是一次寫完所有功能

Spec 只做一件事：讓 AI 不用猜

2.3 為什麼不先寫 Spec，Vibe Coding 一定會歪？

我們來看一些很常見的情境。

情境 A：沒有 Spec 的 Vibe Coding

你對 AI 說：「幫我做一個客戶管理系統」

AI 會怎麼做？

- 自己決定有哪些欄位
- 自己決定畫面長怎樣
- 自己決定流程順序
- 自己幫你加一堆功能

結果通常是：

- 看起來很多東西
- 但你不敢刪
- 也不敢改
- 因為你不知道改了會發生什麼事

情境 B：有 Spec 的 Vibe Coding

你對 AI 說：「我有一個客戶管理功能，資料只有 name / email / phone

流程是：填表 → 新增 → 顯示列表，其他都不要做」

這時候 AI 的狀態會變成：「好，我知道邊界在哪裡了」

結果會是：

- 功能很單純
- 但每一個你都看得懂
- 而且敢改、敢玩、敢延伸

差別不在技術，在於「你有沒有先畫界線」

Spec 的本質不是複雜，而是 幫你先說清楚「做到哪裡就好」。

2.4 在這門課 Spec 的實際用途是什麼？

請你把 Spec 想成三個用途：

1. 防止 AI 自由發揮
2. 防止你自己迷路
3. 防止未來全部重來

尤其第三點，非常重要。

因為 Vibe Coding 最常發生的悲劇是：「東西其實快做完了，但不敢動，只好放棄」，Spec 是讓你「敢繼續」的保險。

2.5 這點一定要釐清：Spec ≠ 一大本文件

你可能現在腦中浮現的是：「那我是不是要寫一份很正式、很厚的文件？」

答案是：完全不是。

在 Week 1，我們的 Spec 只包含三個核心區塊：

Week 1 Spec 的三個組成

區塊	目的	你現在要做到的程度
Schema	定義資料長什麼樣	3–5 個欄位
User Flow	定義順序怎麼跑	5–8 個步驟
Scope	定義做到哪就停	明確寫「不做什麼」

只要這三個有寫清楚，AI 就已經比 80% 的專案好用了。

2.6 為什麼 Spec 要在「寫任何程式之前」完成？

因為一旦你開始寫程式，就會發生三件事：

1. 你會被畫面吸走注意力
2. 你會開始想加功能
3. 你會忘記原本想解決的問題

這時候再回頭寫 Spec，通常只剩兩種結局：

- 懶得寫
- 寫了但不照做

所以這門課硬性規定：Week 1 先寫 Spec，再寫程式

這不是為了折磨你，而是為了讓你後面三週過得比較輕鬆。

2.7 本課程使用的 Spec 種類定位（再對照一次）

這裡我們再用一張表，幫你把「Spec 在哪一層」釘死。

本課程文件層級對照表

層級	文件	問題	你現在的任務
產品層	PRD	我在做什麼產品？	一句話即可
執行層	SDD / Spec	AI 要怎麼做？	今日重點
架構層	DDD	資料不要亂混	理解精神即可

你現在在做的是**執行層**，不是產品策略、也不是系統架構大師。

2.8 一個錯誤觀念，現在就要先拆掉

很多人會以為：「等我功能比較多，再來補 Spec」

這句話在實務上，等於：「等我迷路更久，再來找地圖」

Spec 不是事後補救，是事前導航。

2.9 這一章學完，你應該能做到什麼？

請你自我檢查，如果你現在能做到下面三件事，就代表這一章你過關了：

1. 你知道 Spec 是「給 AI 的操作說明書」
2. 你知道 Week 1 的 Spec 不需要很厚
3. 你知道不先寫 Spec，後面一定會還債

如果你還有點模糊，也沒關係，因為下一章，我們會開始真的動手寫。

下一章你會學什麼？

下一章我們會做一件事：

把腦中混亂的想法，整理成第一份「可用的 Spec 草稿」

你會學會：

- 怎麼從「我想做一個 XXX」
拆成「我現在先做哪一個功能」
- 怎麼決定「哪些東西現在不要做」

Chapter 3 像產品經理一樣整理腦中想法

從「我想做一個 App」到「我現在先做這一個功能」

3.1 先打破一個迷思：你不是要「設計一個完整產品」

很多學員在這個階段，腦中會出現一個壓力源頭：

1. 我是不是要先想好整個產品架構？
2. 如果我現在沒想完整，會不會之後要重來？

先給你一個非常重要的結論：**不會，而且你現在反而不應該想完整。**

在真實世界的產品開發裡，**沒有任何一個產品是「一開始就被想完整的」**。你現在要做的不是「設計產品」，而是**選定第一個可以動手的切入口**。

3.2 產品思維的第一個關鍵能力：降維思考

所謂「降維」，意思是：把一個看起來很大的東西，拆成一個現在就能做的小單元。

舉例來說：

- ❌ 我想做一個專案管理系統
- ❌ 我想做一個學習平台
- ❌ 我想做一個 CRM

這些都太大，大到無法直接開發。

我們要做的是，把它們降到這個層級：

- ✅ 新增一筆專案資料，並顯示出來
- ✅ 新增一筆學習紀錄，並列出列表
- ✅ 新增一個客戶，並看到客戶清單

請注意這個共通點：它們都是一個「動作 + 結果」，這就是你第一個功能該長的樣子。

3.3 所有產品的第一個功能，長得其實都一樣

這是一個對新手來說非常解放的事實：

不管你要做什麼產品，第一個可開發的功能，90% 都是同一種結構

我們把它寫成一句話：「我可以新增一筆資料，然後在畫面上看到它。」，這就是為什麼 Week 1 的課堂示範會選擇「微型 CRM」，不是因為你一定要做 CRM，而是因為這個結構，可以套用到任何產品。

3.4 從「產品想法」到「第一個功能」的拆解步驟

現在我們來實際走一次，你等一下會照這個流程做自己的。

Step 1 | 用一句話描述你的產品 (PRD 極簡版)

請你先完成這一句話，不用完美，只要清楚：

這是一個幫助_____的人，用來_____的工具。

範例：

- 這是一個幫助自由接案者管理客戶的工具
- 這是一個幫助內容創作者追蹤產出進度的工具
- 這是一個幫助學習者記錄學習習慣的工具

這一步的目的不是行銷，是讓你自己知道你在幹嘛。

Step 2 | 問自己一個殘酷但有效的問題

接下來，請你回答這個問題：

如果我今天只能做一個功能
哪一個功能『不存在』，這個產品就完全沒有意義？

這個問題會強迫你放棄 80% 的幻想。

範例：

- CRM：沒有「新增客戶」，其他都不用談
- 學習 App：沒有「記錄一次學習」，其他都沒用
- 內容工具：沒有「新增一篇內容」，什麼都不用做

你現在要選的，就是這個功能。

Step 3 | 把「功能」翻譯成「行為描述」

請避免這種說法：

-  管理客戶
-  追蹤學習狀況

改成這種：

-  新增一個客戶，並在列表中看到
-  新增一次學習紀錄，並顯示紀錄清單

功能一定要能「被執行」，而不是停留在抽象概念

3.5 現在，你的第一個功能應該長這樣

請你試著填寫下面這段話（這會成為你的 Spec 開頭）：

我現在要實作的第一個功能是：使用者可以_____，系統會_____。

範例：

- 使用者可以新增一個客戶，系統會顯示客戶列表
- 使用者可以新增一筆習慣紀錄，系統會顯示歷史紀錄
- 使用者可以新增一個內容任務，系統會顯示任務清單

如果你現在填得出來，代表你已經成功把「想法」降維成「可開發功能」。

3.6 為什麼我們不急著想第二個、第三個功能？

這裡要先幫你踩煞車。

很多人在這個階段會忍不住想：

1. 那我之後要不要加登入？
2. 要不要加統計？
3. 要不要加分享？

先不要。

原因很簡單：你現在還沒有一個「能穩定運作的最小單位」

沒有這個單位，後面加的每一個功能，都只是在增加技術債。

這也是為什麼這門課的設計是：

- W1：Create + Read
- W2：狀態 × Dashboard
- W3：部署 × 商業化

順序不能亂。

3.7 把第一個功能，轉成「Spec 草稿」

現在我們把剛才的思考，正式收斂成一份 **Spec 草稿 (文字版)**。請你照下面格式填寫。

Week 1 | Spec 草稿 (學員填寫版)

功能名稱：

(例：新增客戶)

功能描述：

使用者可以新增一筆_____，系統會_____。

這個功能「不包含」：

(請明確寫不做什麼，例如：不含登入、不含編輯、不含刪除)

你現在不需要寫程式，但你已經完成了一件非常重要的事：

你已經決定「現在要做什麼，不做什麼」

3.8 這一章學完，你現在站在哪？

如果你現在能做到下面三件事，代表你已經準備好進入真正的 SDD：

1. 你選定了一個「第一個功能」
2. 這個功能可以用一句話描述清楚
3. 你知道這個功能目前「不包含什麼」

這就是產品經理在專案一開始最重要的工作。

下一章你會學會什麼？

下一章，我們會開始把這個功能「工程化」：**把你剛剛選定的功能，拆成資料 (Schema)**，你會第一次真的定義：

- 這筆資料長什麼樣
- 有哪些欄位
- 哪些欄位現在不需要

Chapter 4 你的產品資料骨架：Schema

不先定義資料，後面一定全亂

4.1 為什麼我們現在才開始用 AI？

到這一章為止，你已經做了三件很重要的事：

1. 知道你不是在學寫程式
2. 選定了第一個要實作的功能
3. 決定了「現在不做什麼」

現在，你終於有資格開始叫 AI 幹活了。但在那之前，還有一件事一定要先做完：**定義資料長什麼樣 (Schema)**。

4.2 什麼是 Schema

Schema 只有一個意思：「這一筆資料，會包含哪些欄位？」

如果用 Excel 想像：

- 每一欄是什麼？
- 每一列代表什麼？

這就是 Schema。

4.3 為什麼 Schema 沒定義，AI 一定會亂寫？

因為 AI 在沒有 Schema 的情況下，會做三件事：

1. 自己幫你加欄位
2. 用不同名字存同一件事
3. 讓你下一步完全接不起來

這不是 bug，是**規格缺失**。

4.4 Week 1 的 Schema 設計原則 (請照做)

請你嚴格遵守這四個原則：

1. 只用 3–5 個欄位
2. 欄位名稱要清楚、固定
3. 每個欄位都「現在用得到」
4. 不要為了未來先加欄位

4.5 範例：第一個功能的 Schema (示意)

假設你的第一個功能是「新增客戶」：

```
Schema :  
- name  
- email  
- phone  
- createdAt
```

這樣就夠了，真的夠了。

學員專用指令區塊 | Schema 生成

什麼時候用？

當你已經選好「第一個功能」，但不知道資料欄位該怎麼定義時。

為什麼要這樣下指令？

這個指令的目的不是叫 AI 幫你想產品，而是幫你把「一個功能」翻譯成「資料欄位」。

可直接照抄指令 (AWS Q Developer)

你是一個依照 Specification-Driven Development (SDD) 工作的工程協作 AI。

我現在要實作的第一個功能是：

「使用者可以_____，系統會_____。」

幫我定義這個功能需要的最小 Schema，規則如下：

- 欄位數量限制在 3–5 個
- 每個欄位請說明用途
- 不要為了未來功能先加欄位
- 不要加入我沒有提到的概念

用清單格式輸出 Schema。

4.6 你要學會檢查 AI 的 Schema (很重要)

AI 給你 Schema 之後，你要自己問三個問題：

1. 這個欄位現在真的用得到嗎？
2. 有沒有欄位其實是在描述同一件事？
3. 如果我今天不存這個欄位，功能會壞嗎？

如果答案是「不會壞」，請刪掉。

學員專用指令區塊 | Schema 檢查 (必用)

什麼時候用？

當你已經有一份 Schema，但不確定是否太多、太亂。

為什麼要這樣下指令？

避免 AI 自己「優化」成你看不懂的版本。

可直接照抄指令

幫我檢查以下 Schema 是否符合「最小可用原則」：

- 欄位數量是否過多
- 是否有功能重疊的欄位
- 是否有現在用不到的欄位

只提出建議，不要自行修改 Schema。

Schema：

(貼上你的 Schema)

4.7 本章你應該完成的成果

在進入下一章前，請確認你已經完成：

- 一份 3–5 欄位的 Schema
- 你知道每個欄位「現在用來幹嘛」
- 你有用 AI 檢查過 Schema，而不是照單全收

下一章你會學什麼？

下一章，我們會做第二個 SDD 的核心元件：

User Flow：這個功能是「按什麼 → 發生什麼 → 顯示什麼」

而且 Chapter 5 開始，每一章都會有這種「專用指令區塊」。

Chapter 5 讓你的系統「動起來」：User Flow

程式不是魔法，是順序

5.1 為什麼資料定義完，還不能直接寫程式？

到目前為止，你已經完成一件很多新手沒做的事：

- 你知道資料有哪些欄位
- 你知道每一筆資料代表什麼

但這還不夠，因為系統不是資料本身，而是資料怎麼被使用。

換句話說：**資料是靜態的，流程才是讓系統活起來的東西**，而這個「流程」，就是 User Flow。

5.2 什麼是 User Flow？

User Flow 用一句話講完就是：

使用者做了什麼 → 系統做了什麼 → 使用者看到什麼

請注意順序：

1. 先有人（使用者）
2. 再有行為（操作）
3. 最後才有系統反應（畫面或資料）

如果順序錯了，程式一定會怪怪的。

5.3 為什麼沒有 User Flow，Vibe Coding 一定失控？

我們來看一個常見的災難現場。

沒有 Flow 的情況

你對 AI 說：「幫我做一個新增客戶的功能」

AI 可能會：

- 幫你生表單
- 幫你寫 API
- 幫你寫資料庫
- 但你不知道：
 - 什麼時候送資料
 - 成功後畫面要不要刷新
 - 失敗時怎麼處理

結果就是：功能「好像有」，但你不敢用

有 Flow 的情況

你先寫好：

1. 使用者填寫表單
2. 點擊「新增」
3. 系統驗證資料
4. 系統儲存資料
5. 系統重新讀取列表
6. 使用者看到最新資料

這時候 AI 幾乎不可能亂寫，因為你已經把「順序」定死了。

5.4 Week 1 的 User Flow 撰寫原則 (請照做)

這一週的 Flow，請你遵守以下原則：

1. 只寫一條主線流程
2. 不處理例外、不處理錯誤分支
3. 每一步都要是「可執行的動作」
4. 不寫抽象詞 (例如：管理、處理)

你不是在寫教科書，你是在寫「操作說明」。

5.5 範例：微型 CRM 的 User Flow (Create + Read)

這是 Week 1 的標準範例，你之後會一直套用這個結構。

1. 使用者在表單輸入客戶資料
2. 使用者點擊「新增客戶」按鈕
3. 系統讀取表單內容
4. 系統將資料寫入資料庫
5. 系統重新讀取客戶資料
6. 系統更新畫面上的客戶列表

請注意幾件事：

- 沒有登入
- 沒有權限
- 沒有錯誤處理

這不是偷懶，這是刻意降低複雜度。

5.6 把你的 Flow 寫出來

請你現在停下來，把你 Chapter 3 選定的「第一個功能」，照下面格式寫。

Week 1 | User Flow 草稿 (學員填寫版)

1. 使用者：_____
2. 使用者：_____
3. 系統：_____
4. 系統：_____
5. 系統：_____
6. 使用者看到：_____

範例

功能：新增一筆客戶資料 (Create + Read)

1. 使用者：在表單中輸入客戶姓名、Email、電話
2. 使用者：點擊「新增客戶」按鈕
3. 系統：讀取使用者輸入的表單資料
4. 系統：將資料寫入資料庫
5. 系統：重新讀取所有客戶資料
6. 使用者看到：畫面上的客戶列表出現剛剛新增的客戶資料

給學員的小提醒

- 每一步都只能做「一件事」
- 看不見的事 (資料處理) 一定要寫出來
- **第 5 步是新手最常漏掉的關鍵步驟**
- 如果你寫不出第 6 步，代表你還不知道「完成」長什麼樣

為什麼這個範例是「正確的 Week 1 Flow」？

- 有 使用者 → 系統 → 使用者 的完整閉環
- 完全對齊 Create + Read
- 沒有偷加登入、驗證、編輯等高風險元素

- 可以 100% 對應到程式實作，不會卡關

學員專用指令區塊 | User Flow 拆解

什麼時候用？

當你知道「想做什麼功能」，但不知道流程順序怎麼排時。

為什麼要這樣下指令？

這個指令的目的是：**逼 AI 按照「使用者 → 系統 → 畫面」的順序思考，而不是直接生程式碼，要用「行為邊界」與「功能範圍」約束 AI。**

可直接照抄指令

你是一個依照 Specification-Driven Development (SDD) 工作的工程協作 AI。

以下是我目前定義的功能與 Schema：

功能：

使用者可以_____，系統會_____。

Schema：

(貼上你的 Schema)

幫我產出一條「最小可用功能」的 User Flow，規則如下：

- 只包含一個核心功能
- 只寫一條主流程，不包含任何分支
- 步驟數量控制在 5–8 步
- 每一步都必須明確標示是「使用者」或「系統」
- 不處理錯誤、不處理例外、不加入額外功能
- 不包含登入、權限、驗證、編輯或刪除

用條列清單輸出 User Flow。

5.7 Flow 寫完後，你一定要自己檢查的三件事

請你自己對照檢查：

1. 有沒有哪一步「跳太快」？
2. 有沒有一步其實是在做兩件事？
3. 有沒有出現你 Schema 裡沒有的資料？

只要有一個「有」，請回去改。

學員專用指令區塊 | Flow 檢查 (防止 AI 亂補)

什麼時候用？

當你已經有一份 User Flow，但擔心 AI 在理解或回饋時，偷偷幫你加步驟、加功能或腦補情境。

可直接照抄指令

幫我檢查以下 User Flow 是否符合「單一核心功能、主流程清楚」的原則。依序檢查：

1. 是否只包含一條主流程（沒有分支、沒有例外）
2. 是否有跳過中間步驟，或存在未明確寫出的隱含行為
3. 是否有使用到 Schema 中未定義的資料或欄位
4. 是否有加入未被描述的額外功能

只提出問題與改善建議，不要自行重寫或擴充 User Flow。

User Flow：

（貼上你的 User Flow）

Schema：

（貼上你的 Schema）

5.8 本章完成後，你現在擁有什麼？

到這一章結束，你已經有：

- 一份 最小可用 Schema
- 一條 清楚、不亂加戲的 User Flow
- 一組 能約束 AI 的流程指令

這三樣加起來，就是：真正可以拿來 Vibe Coding 的 Spec

下一章你會學什麼？

下一章，我們會做一件很多人會跳過、但超容易出事的事：

把 Schema + Flow，轉成「不會害你重寫整包 code」的 Prompt 結構

也就是：Prompt Engineering (不是聊天，是規格)。

而且 Chapter 6 會是：

- AWS Q Developer × Copilot 角色分工第一次登場
- 指令會變多，但每一條都有用途

Chapter 6 不是聊天，是規格：Prompt Engineering

你不是在拜託 AI，而是在下工程指令

6.1 為什麼你以前下的 Prompt，AI 都「做得很怪」？

我們先來面對一個現實。如果你以前用過這種 Prompt：「幫我寫一個可以新增資料的網站」，那 AI 其實很可憐，因為：

- 它不知道你用什麼技術
- 不知道你資料長什麼樣
- 不知道流程順序
- 不知道做到哪裡要停

於是它只能用「想像力」補齊，而工程場景最怕的就是多餘的想像力。

6.2 在這門課裡，Prompt 的正確定位是什麼？

請你牢記這一句話（之後會一直用到）：

Prompt = 給 AI 的工程規格，不是聊天訊息

也就是說：

- Prompt 不是「請求」
- Prompt 不是「靈感發想」
- Prompt 是「約束條件集合」

你寫得越清楚，AI 能亂來的空間就越小。

6.3 為什麼要先學「結構化 Prompt」？

因為在 Vibe Coding 裡，最容易造成技術債的，不是程式碼本身，而是這兩件事：

1. 一次叫 AI 做太多事
2. 沒有告訴 AI「不要動哪些地方」

這一章，就是在解決這兩個問題。

6.4 Week 1 使用的 Prompt 結構（固定版）

在 Week 1，你只需要學會 一種 Prompt 結構，之後全部套用，我們把它叫做：SDD Prompt 結構。

SDD Prompt 四構面

構面	功能
角色	限定 AI 在什麼位置思考
脈絡	提供已完成的 Spec
任務	明確只做一件事
限制	明確寫「不要做什麼」

只要這四個都有，AI 幾乎不會失控。

6.5 AWS Q Developer × Copilot 的角色分工

在這門課裡，兩個 AI 的角色不一樣。

AWS Q Developer (主力)

你會用它來：

- 根據 Spec 生程式碼
- 重構特定功能
- Debug

你對它的態度是：工程師

GitHub Copilot (副駕)

你會用它來：

- 補齊小段程式
- 自動完成
- 小範圍優化

你不會拿它來下完整需求

學員專用指令區塊 | 建立工程協作角色 (必做)

什麼時候用？

在你 第一次打開專案、準備開始寫任何程式之前。

為什麼要這樣下指令？

先把 AI 的角色「鎖定」，後面它才不會一直換人格。

可直接照抄指令

你是一個依照 Specification-Driven Development (SDD) 工作的資深前端工程協作 AI。

你的任務原則如下：

- 嚴格依照我提供的 Schema 與 User Flow 實作
- 不要自行新增功能
- 不要重構未指定的程式碼
- 每次只處理一個明確任務
- 若資訊不足，請先提出問題，不要自行假設

請回覆「已理解角色與工作方式」。

這一步 只做一次，但非常關鍵。

6.6 把 Schema + Flow 餵給 AI (但不叫它寫 code)

很多新手會犯一個錯：一給 Schema + Flow，就立刻說「幫我寫完整程式」。
請先忍住。正確順序是：先讓 AI「理解」，再讓它「動手」。

學員專用指令區塊 | Spec 對齊確認

什麼時候用？

當你已經完成 Schema 及 User Flow，但還沒開始寫程式。

為什麼要這樣下指令？

避免 AI 誤解你的 Spec。

可直接照抄指令

以下是我目前的 Spec：

功能描述：

使用者可以_____，系統會_____。

Schema：

(貼上你的 Schema)

User Flow：

(貼上你的 User Flow)

先幫我確認：

1. 是否有不清楚或矛盾的地方
2. 是否有 Flow 用到但 Schema 沒定義的資料
3. 是否有超出 Week 1 範圍的內容

只做分析，不要產生任何程式碼。

這一步會幫你省掉大量重寫時間。

6.7 為什麼我們要「一個任務一個 Prompt」？

因為 AI 很容易發生這件事：你叫它改 A，它順手把 B、C、D 全部動掉，這就是 Vibe Coding 技術債的最大來源之一。

學員專用指令區塊 | 單一任務實作 Prompt (核心)

什麼時候用？

當你準備開始實作某一個「明確步驟」。例如：

- 建立表單
- 寫新增資料邏輯
- 顯示列表畫面

為什麼要這樣下指令？

把「可變動範圍」縮到最小。

可直接照抄指令

依照以下 Spec，只完成「一個任務」：

任務：

(例：建立一個可輸入客戶資料的表單 UI)

Schema：

(貼上 Schema)

User Flow (僅供參考)：

(貼上 Flow)

限制條件：

- 只處理上述任務
- 不要新增其他功能
- 不要修改未提及的檔案
- 若需要新增檔案，請先說明原因

請產出對應的程式碼，並簡要說明放在哪個檔案。

6.8 Go Live Server：你一定要「邊做邊看」

請你養成一個習慣：每完成一個小任務，就 Go Live 看畫面。

為什麼？因為畫面是你唯一「不會騙人」的回饋。如果畫面怪怪的？不是你笨，是流程或資料有問題！

6.9 本章你現在應該具備的能力

到這一章結束，你應該已經能做到：

- 知道 Prompt 不是聊天
- 會用固定結構下工程指令
- 知道什麼時候該停下來「只確認、不寫 code」
- 不再一次叫 AI 做一堆事

這代表你已經正式進入：**可控的 Vibe Coding 狀態**。

下一章你會學什麼？

下一章，我們會正式進入：**實作第一個可跑的功能 (Create)**

而且會包含：

- VS Code 專案初始化
- 表單建立
- 資料寫入
- Go Live 即時驗證
- 專用 Debug 指令

Chapter 7 實作第一個可跑的功能（ Create ）

不是寫完再看，而是邊做邊跑

今天的「極簡 CRM」到底包含什麼？

面向	內容
資料	客戶 / 案件基本欄位（ 3-5 個 ）
功能	Create + Read
互動	表單輸入 → 列表更新
技術	前端 + 雲端資料庫
展示	可 Go Live、可 Demo
定位	作品集等級 Demo

沒有的（ 而且刻意不做 ）

面向	為什麼不做
登入權限	你的應用八成會直接炸掉
編輯刪除	Flow 複雜度倍增
搜尋排序	UI 邏輯會亂跑
商業邏輯	Week 2 才做
完整 CRM 流程	這不是今天的目標，今天的目標是先做出一個可用的系統

7.1 現在我們終於要開始「真的做東西」了

走到這一章，你已經完成了很多人會跳過、但其實最重要的準備：

- 你知道你現在要做的是「哪一個功能」
- 你已經定義好 Schema
- 你也寫好 User Flow
- 你知道該怎麼下「不會害你重寫整包」的 Prompt

所以現在這一步非常關鍵：**我們不是要一次做完所有功能，而是要讓「第一個功能真的跑起來」。**

這個「跑起來」，包含兩件事：

1. 程式真的能執行
2. 你在瀏覽器中看得到結果

7.2 Week 1 的實作目標（請牢記）

這一章只做一件事：使用者可以輸入資料，並成功新增一筆資料。先不要想：

- 編輯
- 刪除
- 登入
- 驗證
- 美化 UI

這些現在都不重要，重要的是：你第一次看到「我寫的功能真的有用」的那一刻。

7.3 專案初始化 (VS Code)

在開始寫任何程式之前，請先完成以下檢查：

- 已安裝 VS Code
- 已啟用 AWS Q Developer 擴充套件
- 已啟用 GitHub Copilot
- 已安裝 Go Live Server (Live Server)

請先建立一個全新的專案資料夾，例如：`vibe-coding-week1`，然後在 VS Code 打開它。

學員專用指令區塊 | 專案初始化協作指令

什麼時候用？

當你剛建立好專案資料夾，準備請 AI 幫你規劃最基本的檔案結構時。

為什麼要這樣下指令？

避免 AI 一開始就幫你生一堆你看不懂、也用不到的檔案。

可直接照抄指令

你目前正在協助我建立 Week 1 的最小可跑專案。

依照以下原則，幫我建議最基本的前端檔案結構：

- 技術為純 HTML / CSS / JavaScript
- 不使用任何框架
- 目標是支援一個「新增資料」功能
- 可以用 Live Server 直接執行

只提供必要的檔案清單與用途說明，不要先產生完整程式碼。

你應該只會看到非常精簡的結構，這是對的。

7.4 建立輸入介面 (表單)

你的第一個畫面，只需要一件事：讓使用者可以輸入資料，畫面不用漂亮，但一定要清楚。

學員專用指令區塊 | 建立輸入表單 (Create UI)

什麼時候用？

當你已經有專案結構，準備開始做第一個「看得到的畫面」。

為什麼要這樣下指令？

避免 AI 一次幫你「整個系統」都寫出來。

可直接照抄指令

依照以下 Spec，幫我建立一個「新增資料」的表單 UI。

功能描述：

使用者可以新增一筆資料。

Schema：

(貼上你的 Schema)

限制條件：

- 只需要輸入欄位與送出按鈕
- 不需要樣式美化
- 不需要資料驗證
- 不要加入任何其他功能

產出對應的 HTML，並說明放在哪個檔案。

7.5 用 Go Live Server 驗證畫面

請你立刻做這件事：

- 在 VS Code 中
- 進入 index.html
- 點選右下方 **Go Live**

你應該會看到瀏覽器出現：

- 一個可以輸入資料的表單
- 沒有錯誤畫面
- 可以正常顯示在瀏覽器

如果這一步你看到了畫面，恭喜你，你已經成功一半了。

7.6 寫入資料的邏輯 (Create Logic)

現在我們要做的是：把使用者輸入的資料，真的「存起來」。

在 Week 1，你只需要先做到：

- 能成功寫入資料
- 不用考慮錯誤處理
- 不用考慮最佳化

學員專用指令區塊 | 新增資料邏輯 (Create)

什麼時候用？

當你的表單已經能顯示，準備讓它「真的有作用」。

為什麼要這樣下指令？

避免 AI 幫你順便加一堆你現在還不需要的東西。

可直接照抄指令

請依照以下 Spec，幫我完成「新增資料」的 JavaScript 邏輯。

Schema：

(貼上你的 Schema)

User Flow：

(貼上你的 User Flow)

限制條件：

- 只處理資料收集與寫入
- 不處理錯誤情況
- 不新增其他功能
- 不重構未提及的程式碼

請說明需要新增或修改哪些檔案，並提供對應程式碼。

7.7 再次 Go Live：確認資料真的有被新增

完成後，請你再 Go Live 一次，此時你應該可以做到：

- 輸入資料
- 點擊新增
- 沒有報錯
- 功能流程是完整的

就算你還看不到列表，也沒有關係，能新增成功，本身就是一個完整的里程碑。

7.8 本章完成後，你現在擁有什麼？

到這裡，你已經完成了：

- 第一個 UI
- 第一段可用的程式邏輯
- 第一個「真的會動」的功能

這代表：你已經成功跨過「我只是在看 AI 表演」，進入「我在指揮 AI 做事」的狀態了！

下一章你會學什麼？

下一章，我們會把這個功能補完整：

Read：把你新增的資料，顯示成列表

你會第一次看到：

- 資料真的被存起來
- 畫面真的會更新
- 系統開始「像產品」

附錄：Debug

新手最常壞在這裡，壞掉不是失敗，是系統在跟你說話求救

D-1 | 先說一句很重要的話

如果你現在的狀態是：

- 畫面有出來
- 但按鈕按了沒反應
- 或 Console 出現紅字
- 或你根本不知道現在到底算不算成功

請你先放心一件事：

你現在遇到的，不是錯誤，而是「第一次真的在做系統」的正常現象

這一節的目的不是讓你變成 Debug 專家，而是讓你學會三件事：

1. 不要慌
2. 知道問題大概落在哪一類
3. 知道「該怎麼請 AI 幫你修，而不是害它亂修」

D-2 | 新手最常見的 5 種壞法

壞法 1：畫面正常，但按了完全沒反應

典型情況

- 表單有顯示
- 按鈕可以按
- 但資料沒有新增
- Console 沒有任何訊息

最常見原因

- submit 事件沒有綁到
- JavaScript 檔案沒有被正確引入
- id / name 對不到

這不是邏輯錯，是「線沒接好」

壞法 2：Console 一堆紅字，看不懂

典型情況

- 一開頁面就爆紅
- 或一按按鈕就紅一片

最常見原因

- 變數名稱打錯
- 存取不存在的欄位
- JavaScript 執行順序錯誤

紅字不是罵你，是在指路

壞法 3：資料好像有存，但畫面沒更新

典型情況

- 沒有報錯
- 你「感覺」資料有進去
- 但畫面完全沒變

最常見原因

- 寫入成功後，沒有重新讀資料
- 或沒有更新 DOM

這是 Flow 漏了一步

壞法 4：AI 幫你改了一堆你沒叫它改的東西

典型情況

- 本來只有一個功能壞
- 修完之後，其他地方一起壞

最常見原因

- Debug Prompt 太模糊
- 沒有限制「只能修哪裡」

這是 Prompt 問題，不是你技術不好

壞法 5：你不知道現在該不該算成功

典型情況

- 功能好像有一點點動
- 但不確定是不是完成了

這代表你還沒定義「成功條件」

D-3 | Week 1 Debug 的唯一正確心法

在這一週，請你牢記這三句話：

1. 一次只檢查一件事
2. 不要自己亂改，先確認問題在哪
3. 請 AI 修「點」，不要修「面」

你現在不是在比誰厲害，你是在保護你的專案不要長出技術債。

學員專用指令區塊 | 定位問題

什麼時候用？

當功能壞掉，但你不知道是哪裡壞。

為什麼要這樣下指令？

避免 AI 一開始就幫你「全部重寫」。

可直接照抄指令

我目前遇到問題，但不確定問題點在哪。以下是狀況描述：

- 我原本預期的行為是：_____
- 實際發生的狀況是：_____
- 是否有錯誤訊息 (Console) : (有 / 沒有，若有請貼上)

先幫我判斷：

1. 問題比較可能是出在 UI、事件綁定、還是資料處理
2. 建議我「先檢查哪一個地方」

只做問題定位，不要直接修改程式碼。

學員專用指令區塊 | 局部修正 (救命指令)

什麼時候用？

當你已經知道「大概是哪一段壞了」。

為什麼要這樣下指令？

防止 AI 幫你順手改掉其他正常功能。

可直接照抄指令

只協助我修正以下這一個問題，不要動其他功能。

問題描述：

(簡要描述問題)

相關檔案：

(貼上檔名與相關程式碼)

限制條件：

- 只修改必要的行數
- 不重構其他部分
- 不新增新功能

說明你改了哪裡，以及為什麼要這樣改。

學員專用指令區塊 | Flow 回溯檢查 (防止亂修)

什麼時候用？

當你發現「修來修去，好像更亂」。

可直接照抄指令

以下是我目前的 User Flow：

(貼上 Flow)

幫我對照目前的程式行為，檢查是否有：

- 某一步沒有被實作
- 某一步順序錯誤
- 某一步被跳過

指出 Flow 與實作不一致的地方即可，不要重寫程式。

D-4 | 什麼情況下，你可以說「我這一關過了」？

在 Week 1，Debug 成功的標準不是「完全沒 bug」。而是你可以說出這句話：**我知道現在壞在哪一類，而且我知道下一步該怎麼查。**

只要你能做到這件事，你就已經比 90% 只會照抄的人更進階了。

Chapter 8 把資料顯示成列表：Read

系統第一次「看起來像產品」

8.1 為什麼 Read 這一步這麼重要？

很多人以為資料有存就好，之後再顯示也沒差吧？實際上，Read 才是你第一次真正確認系統是完整的地方。

因為只有在你能做到下面三件事時，你才真的做出了一個「可用的功能」：

1. 資料能被新增
2. 資料能被讀出
3. 使用者能在畫面上看到結果

如果少了 Read，你現在做的還只是「半套」。

8.2 Week 1 的 Read 在做什麼（請不要擅自加戲）

在這一週，Read 的目標非常單純：把所有已存在的資料，顯示成一個列表。

請你再次確認，這一章「不做」以下事情：

- 不排序
- 不分頁
- 不搜尋
- 不編輯
- 不刪除

我們要的是「穩定」，不是「完整」

8.3 什麼是 Read 的本質：不是畫面，是「資料流動」

請你先建立一個正確的理解：**Read 不是把畫面畫出來，而是把「資料 → 畫面」這條線接起來**，這條線如果沒接好，後面所有功能都會怪怪的。

8.4 Read 的基本流程（人話版）

請你對照你的 User Flow，Read 一定包含這幾步：

1. 系統讀取所有資料
2. 系統將資料轉成可顯示的結構
3. 系統把資料渲染到畫面上

如果你現在的程式「感覺很亂」，通常是這三步混在一起寫。

學員專用指令區塊 | 建立資料列表容器

什麼時候用？

當你已經完成新增資料，但畫面上還沒有地方可以顯示資料時。

為什麼要這樣下指令？

先把「顯示區域」固定下來，避免後面一直改 UI。

可直接照抄指令

依照目前的功能，幫我在畫面中建立一個「資料列表顯示區」。

限制條件：

- 只需要基本的列表結構（例如 `ul / div`）
- 不需要任何樣式
- 不要加入互動功能

說明需要新增或修改哪個 HTML 區塊。

8.5 讀取資料的邏輯 (Read Logic)

現在我們要做的事是：在頁面載入時，把已存在的資料全部讀出來。這一步非常重要，因為它決定了：

- 使用者重新整理頁面時，看不看得到資料
- 系統是不是「真的有存東西」

學員專用指令區塊 | 讀取資料邏輯 (Read)

什麼時候用？

當你已經可以新增資料，但還沒辦法顯示資料列表。

為什麼要這樣下指令？

避免 AI 把 Read 跟 Create 混在一起寫，導致流程混亂。

可直接照抄指令

依照以下 Spec，幫我完成「讀取資料並顯示列表」的邏輯。

Schema：

(貼上你的 Schema)

User Flow (Read 相關)：

- 系統讀取所有資料
- 系統更新畫面上的資料列表

限制條件：

- 只處理資料讀取與畫面顯示
- 不加入排序、搜尋或其他功能
- 不修改新增資料的邏輯

說明需要新增或修改哪些 JavaScript 程式碼。

8.6 當新增資料後，列表要怎麼更新？

這裡是新手**第二個高風險區**。你有兩種做法，但在 Week 1，請選最穩的那一個。

做法 A (建議)

- 新增成功後
- 重新呼叫「讀取資料」的函式
- 更新整個列表

做法 B (先不要)

- 自己手動把新資料插進 DOM

做法 B 在 Week 1 很容易亂掉，我們先不要。

學員專用指令區塊 | 新增後刷新列表 (穩定版)

什麼時候用？

當你發現「資料有新增，但列表沒有更新」。

可直接照抄指令

目前我已經完成：新增資料及讀取資料並顯示列表。協助我在「新增成功後」，重新觸發資料讀取流程，以確保畫面列表會更新。

限制條件：

- 不改寫原本的讀取邏輯
- 不新增其他功能

指出需要在哪裡呼叫既有函式即可。

8.7 Go Live Server：確認「完整流程」跑通

現在請你再 Go Live 一次，確認以下流程可以完成：

1. 開啟頁面
2. 看到既有資料列表
3. 輸入資料
4. 點擊新增
5. 列表更新，出現新資料

如果這五步都能跑完：恭喜你，你已經完成了一個完整的 CRUD (Create + Read) 循環。

8.8 本章完成後，你現在站在哪？

請你回頭看一件事：一個小時前，你可能還在想「我真的做得出一個系統嗎？」，但現在，你手上已經有：

- 一個可以新增資料的介面
- 一個會顯示資料的列表
- 一個真的跑得起來的網站

這不是玩具，這是**產品雛形**。

在進入下一章之前，你需要先知道一件事

如果你現在心裡出現這個想法：資料能新增、也能顯示，那是不是代表系統已經完成了？

答案是：功能完成了，但「產品還沒站穩」。

下一章，我們不會立刻加新功能，而是會先幫你做一件非常重要的事：**確認你現在做出來的東西，是不是一個「可以繼續擴充，而不會一路還債」的基礎。**

下一章你會學什麼？

你會學到：

- 為什麼現在「不加功能」反而是對的
- 哪些事情現在做，會變成技術債
- 你現在的系統，哪些地方已經夠穩
- 接下來六天，該怎麼打磨而不亂改

這一章會幫你「做出來的成就感」，轉成「可以放心往下走的安全感」。

Chapter 9 你現在擁有的是「可擴充的雛形」

而不是「還沒完成的半成品」

9.1 先幫你正名一件事

如果你現在心裡有一個念頭是：好像還缺很多功能？這樣真的算完成一個系統嗎？

請你先把這個想法放下，因為你現在完成的不是「玩具」，而是：

一個可以穩定運作、可以繼續擴充的產品雛形

這兩者差非常多。

9.2 回顧你在 Week 1 真正完成的事情

請你慢慢對照下面這張表。

Week 1 完成項目盤點

項目	狀態	說明
第一個功能定義		你知道自己在做什麼
Schema 定義		資料不再亂存
User Flow		系統行為有順序
Create 功能		資料真的能新增
Read 功能		資料真的能顯示
Go Live 驗證		系統真的能跑

如果你這張表大多數是打勾的，代表你已經跨過新手最大的一道牆。

9.3 你現在「刻意沒有做」的事情（這是好事）

請你注意，下面這些不是你「來不及做」，而是你現在不應該做：

- 登入 / 權限
- 編輯 / 刪除
- 搜尋 / 排序
- 表單驗證
- 美化 UI

這些事情現在做，會發生一件事：你會在還沒站穩之前，開始欠技術債。

9.4 今日最常見的技術債來源（請避開）

技術債類型 1：功能堆疊太快

常見狀況

- Create 剛跑起來
- 就想加 Edit、Delete、Login

為什麼是債？

- Schema 還沒穩
- Flow 還沒驗證
- 一改就連環壞

技術債類型 2：沒有回頭對齊 Spec

常見狀況

- 改程式改得很順
- 但 Spec 完全沒更新

後果

- 你自己都不知道現在系統在幹嘛
- AI 更不知道

技術債類型 3：讓 AI 自由重構

常見狀況

- 「幫我優化一下」
- AI 幫你重寫半包

後果

- 你看不懂
- 壞了不敢修

9.5 Week 1 正確的進階姿勢（先知道就好）

你現在不需要馬上做，但可以先知道：

- Edit / Delete 是 Schema × Flow 的延伸
- 登入是「另一個系統」
- UI 美化應該最後才做
- 商業化要回到 Spec

這些都會在後面的課程慢慢來。

9.6 如果你現在想繼續改，請先問自己三個問題

在課後六天，如果你想加東西，請先停下來，問自己：

1. 這個功能有寫進 Spec 嗎？
2. 這個功能是「現在必須」嗎？
3. 加了它，會不會讓原本功能變不穩？

只要有一個答案是「不確定」，請先不要做。

學員專用指令區塊 | 技術債風險檢查

什麼時候用？

當你想在 Week 1 結束後，繼續加功能。

為什麼要這樣下指令？

幫你判斷「這是進步，還是技術債」。

可直接照抄指令

以下是我目前已完成的功能與對應規格 (Spec):(貼上你的 Spec)

我正在考慮新增以下功能:(描述你想新增的功能)

只協助我進行「開發風險評估」，並回答以下問題：

1. 這個功能是否適合在目前的系統成熟度下加入？
2. 新增此功能，可能會造成哪些技術債或結構風險？
3. 從開發順序的角度來看，這個功能較適合：
 - 現在就加入
 - 等核心功能更穩定後再加入
 - 等整體架構調整完成後再加入

只提供分析與建議，不要產生任何程式碼或實作方案。

9.7 這一章你現在應該帶走的觀念

請你把下面這句話記住：

現在不做的，不是放棄
而是為了之後可以走更遠

你現在站的位置，是一個非常好的起點。

下一章你會學什麼？

最後一章，我們會把 Week 1 真正「收尾」：把這一套流程，套用到你自己的產品想法上。而且會包含：

- 課後作業說明
- 個人產品延伸任務
- 為 Week 2 做準備

Chapter 10 課後作業與個人產品延伸

把「會做」變成「真的在做」

10.1 先說清楚：這不是交作業，是產品啟動

在 Week 1 結束時，我們不會用「你寫了多少 code」來判斷你有沒有學會。真正的標準只有一個：

你能不能把同一套流程，套用到「你自己想做的產品」上

如果你可以做到這件事，代表你已經掌握了這門課的核心能力。

10.2 課後作業的真正目的

這份課後作業不是要你做得多完整，而是要確認三件事：

1. 你能不能獨立寫出一份「最小 Spec」
2. 你能不能用 Spec 指揮 AI，而不是拜託 AI
3. 你能不能控制住自己，不亂加功能

只要這三件事成立，Week 1 就是成功的。

10.3 課後作業內容說明 (請一步一步來)

你的課後作業，會沿用 **Week 1** 的完整流程，但換成你的產品想法。

作業任務一 | 定義你的「第一個功能」

請你用一句話完成下面這段描述：

我現在要實作的第一個功能是：使用者可以_____，系統會_____。

請注意：

- 只能一個功能
- 不要寫成願景
- 不要偷塞第二個功能

作業任務二 | 完成你的 Spec (文字版)

你的 Spec 必須包含以下三個區塊：

Schema (3-5 個欄位)

- 欄位名稱
- 欄位名稱
- 欄位名稱

User Flow (5-8 步)

1. 使用者：
2. 使用者：
3. 系統：
4. 系統：
5. 使用者看到：

Scope (明確寫不做什麼)

- 不包含登入
- 不包含編輯
- 不包含刪除

作業任務三 | 實作 Create + Read

請你完成以下功能：

- 可以新增一筆資料
- 可以在畫面上看到資料列表
- 可以用 Go Live Server 正常操作

不需要：

- 美化
- 驗證
- 最佳化

學員專用指令區塊 | 個人產品 Spec 啟動指令

什麼時候用？

當你準備把 Week 1 的流程，套用到你自己的產品。

可直接照抄指令

我想將「單一核心功能 → 規格 → 實作」的流程，套用到我自己的產品上。

產品描述：這是一個幫助_____的人，用來_____的工具。目前我只專注於「第一個核心功能」：使用者可以_____，系統會_____。

協助我完成以下兩件事：

1. 定義一份「最小可用 Schema」

- 欄位數量限制在 3–5 個
- 每個欄位請簡要說明用途
- 不為未來功能預留欄位

2. 產出對應的 User Flow

- 只包含一條主流程
- 步驟數量控制在 5–8 步
- 每一步需明確標示是「使用者」或「系統」

遵守以下限制條件：

- 不新增任何額外功能
- 不加入登入、權限、驗證、編輯或刪除
- 不處理錯誤或例外情境
- 不假設未提及的使用情境

先輸出 Schema，再輸出 User Flow。

10.4 課後六天，你該怎麼安排才不會爆掉？

我非常建議你用下面這個節奏，而不是一口氣狂改。

課後六天建議節奏

天數	建議任務
Day 1	寫好 Spec (不要寫程式)
Day 2	Schema + Flow 檢查
Day 3	完成 Create
Day 4	完成 Read
Day 5	Debug 與穩定
Day 6	整理成果，準備進入 Week 2

如果你覺得某一天「卡住了」，是正常的，請不要硬撐。

學員專用指令區塊 | 卡關救援指令

什麼時候用？

當你在課後作業中卡住，不知道該不該繼續改。

可直接照抄指令

我正在進行課後作業，目前遇到卡關狀況，想請你協助我做「行動判斷」，而不是寫程式。

目前狀態如下：

- 已完成的項目：

（條列說明，例如：已完成 Spec / 已完成 Schema / 已完成 Create 但尚未完成 Read）

- 目前卡住的地方：

（具體描述卡關點，例如：流程接不起來 / 不確定要不要加功能 / 畫面結果不符合預期）

請你協助我判斷以下兩件事：

1. 以目前狀態來看，我比較適合「繼續修改現有內容」，還是「先停下來整理或修正 Spec」
2. 在不增加新功能、不擴大範圍的前提下，我下一個最小且安全的行動是什麼

只提供判斷與行動建議，不要產生任何程式碼，也不要幫我新增功能。

10.5 什麼狀況下，你可以安心進入 Week 2？

如果你符合下面三個條件，請放心，你已經準備好了：

1. 你的產品有一個 **穩定可用的 Create + Read**
2. 你知道每一筆資料為什麼存在
3. 你敢改、也知道壞了怎麼救

這時候再進入 Week 2，你會非常穩。

10.6 Week 1 的真正收穫（請帶走這句話）

在 Week 1，你真正學到的不是工具，而是這件事：

我可以用規格思維，讓 AI 幫我把想法變成可以跑的產品

這個能力，會在接下來兩週一直被放大。

下一步，你會在 Week 2 做什麼？

Week 2，我們會在你已經站穩的基礎上，開始做兩件事：

- 狀態管理（系統開始變「聰明」）
- Dashboard（開始像真的產品）

但那之前，請你先好好完成 Week 1 的課後任務。

附錄：一頁判斷指南

什麼時候該停下來，而不是繼續寫

這一頁不是阻止你進步，而是幫你避免「努力方向錯誤」造成的技術債。

一、先給你一個最重要的總原則

請你記住這一句話：只要你現在「不知道自己在改什麼」，就不應該繼續寫程式。

停下來，不是放棄，而是把方向校正好再前進。

二、出現以下狀況時，請「立刻停下來」

狀況 1 | 你一直在改，但說不清楚「為什麼要改」

典型內心旁白

- 好像怪怪的，再試試看
- AI 說可以這樣改
- 先跑起來再說

判斷

- 你現在不是在實作
- 你是在碰運氣

行動

- 停下來
- 回到 Spec
- 用一句話寫出：「我現在要解決的問題是_____」

狀況 2 | 你開始想加新功能來「順便解決問題」

典型行為

- 原本是修 Create
- 卻開始想加 Edit

- 或想加驗證、登入、優化

判斷

- 這不是解決問題
- 這是在逃避問題

行動

- 停下來
- 問自己：「這個新功能，有寫在 Spec 裡嗎？」

如果沒有，先不要做。

狀況 3 | 你不確定現在到底算不算成功

典型感覺

- 好像有一點動
- 但不知道對不對
- 不知道該不該繼續

判斷

- 你沒有定義完成條件

行動

- 停下來
- 補一句話到 Spec：「當_____發生時，這個功能就算完成」

狀況 4 | 你不敢再改，因為怕整包壞掉

典型行為

- 能動，但不敢碰
- 不知道哪裡能改、哪裡不能改

判斷

- 技術債正在生成中

行動

- 停下來
- 回到 Flow

- 一步一步對照：哪一步有實作？哪一步沒有？

狀況 5 | 你已經不知道「下一步要做什麼」

典型狀態

- 看著專案發呆
- 不知道是要寫、要修、還是要加

判斷

- 你缺的是「下一個最小行動」，不是能力

行動

- 停下來
- 問自己：「如果只做一件最小的事，會是哪一件？」

三、什麼時候「可以放心繼續寫」？

你可以繼續寫的前提只有一個：你可以完整回答這三句話

1. 我現在要處理的是哪一個功能
2. 這個修改對應 Spec 的哪一段
3. 改完之後，我要檢查什麼結果

只要有一個答不出來，請先停。

四、給學員的最重要提醒

停下來不是浪費時間，沒有方向地繼續寫，才是

附錄：課後救援指令卡

卡關時，不要硬寫，先用這些指令

這一組指令的目的不是幫你寫程式，而是幫你「決定接下來該做什麼」。

指令卡 1 | 我要不要先停下來整理 Spec？

我正在進行課後作業，目前有點卡關，想請你協助我判斷方向，而不是寫程式。

目前狀態：

- 已完成的項目：(條列說明)
- 卡住的地方：(具體描述)

請協助我判斷：

1. 我現在比較適合繼續修改現有內容，還是先停下來整理或修正 Spec
2. 做出這個判斷的理由是什麼

只給判斷與理由，不要產生任何程式碼。

指令卡 2 | 我現在的下一個最小行動是什麼？

我不確定接下來該做什麼，請你協助我找出下一個「最小且安全」的行動。

目前狀態：

- 已完成的項目：
- 目前目標功能：

請你在不新增功能、不改架構的前提下，建議我下一個最小行動是什麼，以及完成它的判斷標準。

不要產生程式碼。

指令卡 3 | 我是不是在製造技術債？

我想請你協助我判斷，我現在的修改方向是否有技術債風險。目前狀況：

- 我原本要解決的問題是：
- 我現在正在做的修改是：

請你協助我判斷：

1. 這個修改是否偏離原本的 Spec
2. 是否有技術債風險
3. 是否建議我先停下來整理思路

只做評估，不要提供程式碼。

指令卡 4 | 我現在是不是想太多了？

我懷疑自己現在有點過度思考，請你幫我做一次簡單的狀態校正。

目前狀況：

- 已完成的功能：
- 我現在想加的東西：

請你協助我判斷：

1. 這個想加的內容是否現在必要
2. 是否可以延後到後續階段
3. 我現在最該專注的一件事是什麼

不要產生程式碼。

真正會做產品的人，不是一直寫的人，而是知道什麼時候該停下來的人。